

LeMETER

Ahmed Maruf and Luca Natilla

Computer Vision 2025/2026

Overview

1. Monocular Depth Estimation
2. Dataset
3. Theoretical intro to LeJEPA
4. Setup
5. Ablation
6. Probing and Metrics
7. Results
8. Conclusion
9. Limitations
10. Gaussian METER

1. Monocular Depth Estimation

Monocular Depth Estimation (MDE)

- Ill-posed problem: we want to recover information about 3D world from 2D image
- Regression problem



State of the Art

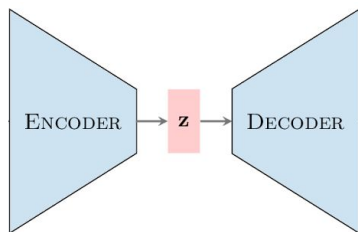
2014 — Eigen et al. introduce the first end-to-end CNN for single-image depth prediction

2019 — First wave of purpose-built CNN based lightweight models, aimed at embedded/robotics deployment.

- Wofk et al., FastDepth [8]
- Papa et al., SPEED [6]

2022 — Hybrid CNN+Transformer lightweight architectures emerge

- Zhang et al. Lite-Mono [9]
- Papa et al., METER [1]



2025 — Newer efficient-architecture directions, moving past plain CNN/ViT toward state-space models and multi-order feature fusion:

- LMDepth [3]
- EfficientDepth [10]

$$h'(t) = \mathbf{A}h(t) + \mathbf{B}x(t),$$

$$y(t) = \mathbf{C}h(t).$$

$$\overline{\mathbf{A}} = \exp(\Delta\mathbf{A}),$$

$$\overline{\mathbf{B}} = (\Delta\mathbf{A})^{-1}(\exp(\Delta\mathbf{A}) - \mathbf{I}) \cdot \Delta\mathbf{B}.$$

Discretization via ZOH

$$h_t = \overline{\mathbf{A}}h_{t-1} + \overline{\mathbf{B}}x_t,$$

$$y_t = \mathbf{C}h_t.$$

Contribution

- Investigate if latent space engineering can boost accuracy or training efficiency
- Assess if LeJEPa framework can be applied to regression tasks too
- Analyze the difference between **invariance** and **equivariance**
- Gaussian METER: a more accurate or more efficient METER with zero overhead cost

2. Dataset

Dataset

NYU Depth v2 from Kaggle <https://www.kaggle.com/datasets/soumikrakshit/nyu-depth-v2>

- 50k train set
- 654 test set

Dataset link provided by AdaBins does not work anymore.

Indoor images taken with Microsoft Kinect. The full dataset is made out of frames of a long video. To avoid correlation between input samples, we use a portion of the full dataset (the 50k split).

3. Theoretical Intro to LeJEPa

LeJEPA

- Latent
- Euclidean
- Joint
- Embedding
- Predictive
- Architecture

arXiv:2511.08544v3 [cs.LG] 14 Nov 2025

LeJEPA: Provable and Scalable Self-Supervised Learning Without the Heuristics

Randall Balestriero^{1,2,*} Yann LeCun^{3,2,*}

¹ Brown University ³ New York University (NYU) ² Meta-FAIR

* Equal contribution

Learning manipulable representations of the world and its dynamics is central to AI. Joint-Embedding Predictive Architectures (JEPAs) offer a promising blueprint, but lack of practical guidance and theory has led to ad-hoc R&D. We present a comprehensive theory of JEPAs and instantiate it in **LeJEPA**, a lean, scalable, and theoretically grounded training objective. First, we identify the isotropic Gaussian as the optimal distribution that JEPAs' embeddings should follow to minimize downstream prediction risk. Second, we introduce a novel objective—**Sketched Isotropic Gaussian Regularization (SIGReg)**—to constrain embeddings to reach that ideal distribution. Combining the JEPA predictive loss with SIGReg yields LeJEPA with numerous theoretical and practical benefits: (i) single trade-off hyperparameter, (ii) linear time and memory complexity, (iii) stability across hyper-parameters, architectures (ResNets, ViTs, ConvNets) and domains, (iv) heuristic-free, e.g., no stop-gradient, no teacher-student, no hyper-parameter schedulers, and (v) distributed training-friendly implementation requiring only ≈ 50 lines of code. Our empirical validation covers 10+ datasets, 60+ architectures, all with varying scales and domains. As an example, using imagenet-1k for pretraining and linear evaluation with frozen backbone, LeJEPA reaches 79% with a ViT-H/14. We hope that the simplicity and theory-friendly ecosystem offered by LeJEPA will reestablish self-supervised pre-training as a core pillar of AI research ([GitHub repo](#)).

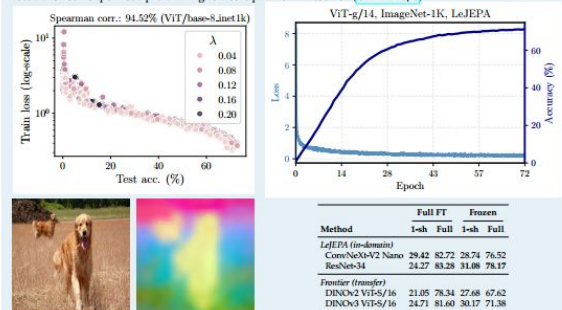


Figure 1. LeJEPA overview. Top-left: Training loss exhibits strong correlation with downstream linear probe performance on ImageNet-1K (ViT-base), providing the first practical loss for model selection without supervised probing. Top-right: Training stability without heuristics even on 1.8B ViT-g models, stable training loss. Bottom-left: PCA features from ImageNet-1K pretrained LeJEPA ViT-Large demonstrate clear semantic relationships. Bottom-right: Galaxy 10 in-domain results showcasing LeJEPA's in-domain pretraining consistently outperforms state-of-the-art frontier foundation models transfer learning (DINOv2/v3 trained on natural images) across data regimes from 1-shot to full supervision. This demonstrates that domain-specific SSL beats generic transfer learning, even against massive-scale frontier models, when the framework scales effortlessly to any domain, model, and data scale.

Latent Space Properties

Latent space has two desirable properties:

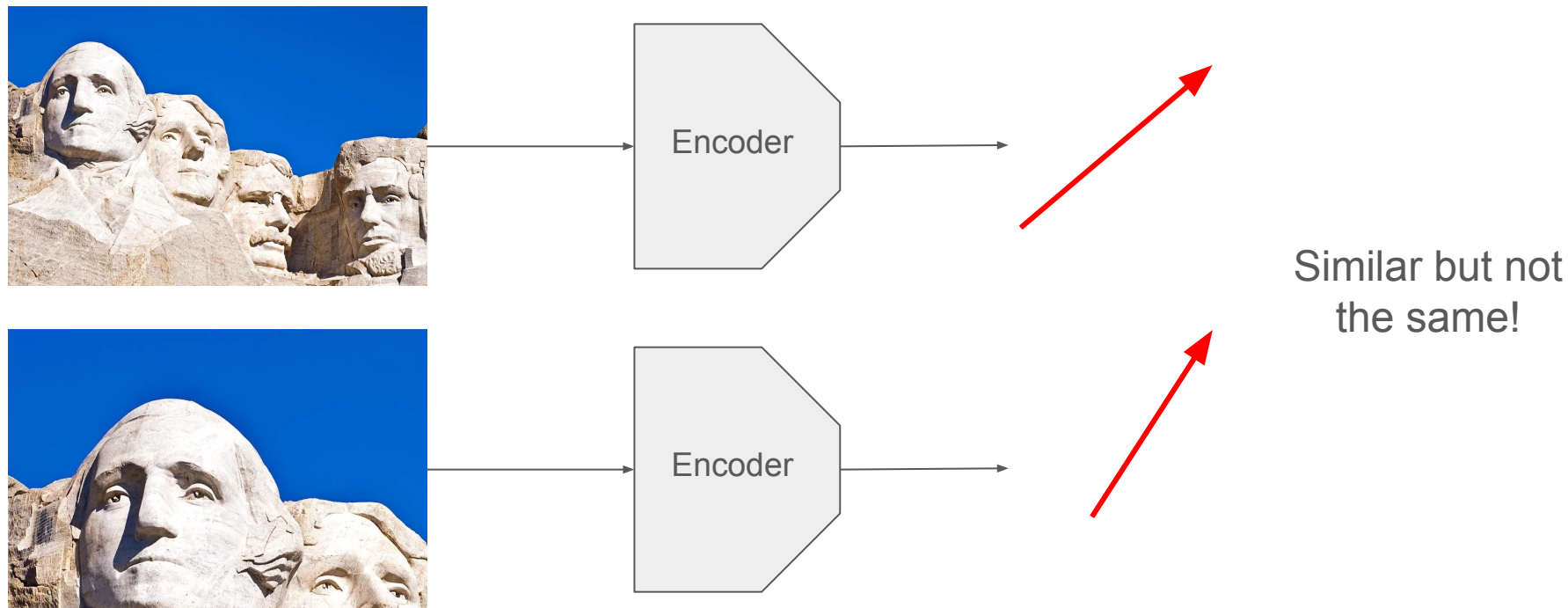
1. **View Invariance:** “two views of the same thing should produce similar embeddings”. We want to be able to predict the embedding of one view $x_{n,v}$ from another view $x_{n,v'}$

$$\text{Pred}(x_{n,v}) \approx \text{Pred}(x_{n,v'})$$

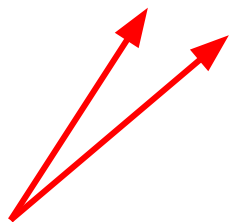
2. **Non-degenerate Latent Space:** embedding must preserve an informative and rich structure. We want to avoid representation collapse. “Do not encode everything as the same identical vector”

$$x_i \neq x_j$$

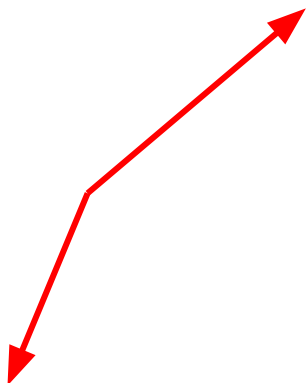
Embeddings: Non-degenerative Invariance



Embeddings: Non-degenerative Invariance

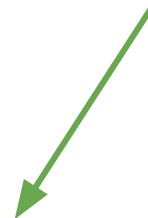
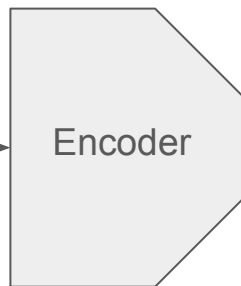
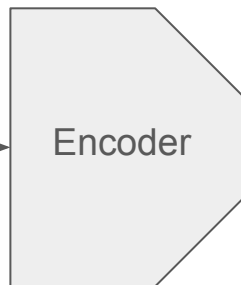


We want this



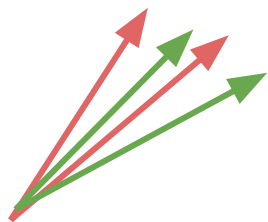
Not this!

Embeddings: Non-degenerative Invariance

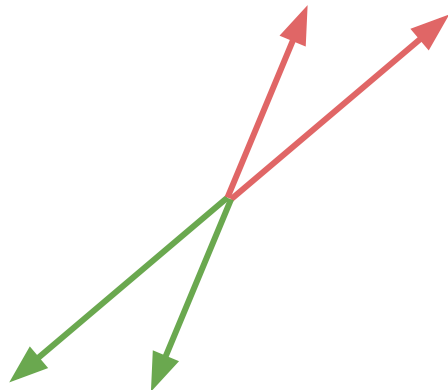


Not similar and
not the same!

Embeddings: Non-degenerative Invariance



We don't want this!



We want this

Isotropic Gaussian

$$\Sigma = I\sigma^2$$

- One scalar variance, no covariance between dimensions (identical spread in every direction)
- Cuts covariance parameters from $O(k^2) \rightarrow O(1)$, avoiding the cost of estimating a full $k \times k$ matrix in high-dim embedding spaces
- **Why it matters for SSL**: encourages embeddings to spread across the full space and stay decorrelated, preventing representation collapse
- **Mathematically optimal for linear probing [1]**

Sufficient Condition for Gaussianity (Cramér–Wold)

Given data in $\mathbb{R}^D$, if the projection onto every unit vector on the hypersphere $\mathbb{S}^{D-1}$ forms a 1-dimensional Gaussian distribution, then the data itself follows a D-dimensional Gaussian distribution.

More formally:

Let X be a random vector in $\mathbb{R}^D$.

If, for every unit vector $a \in \mathbb{S}^{D-1}$ (i.e. $\|a\| = 1$),

$$\langle a, X \rangle \sim \mathcal{N}(\mu_a, \sigma_a^2) \quad (\text{a 1-D Gaussian})$$

then

$$X \sim \mathcal{N}(\mu, \Sigma) \quad (\text{a D-dimensional Gaussian})$$

for some mean vector $\mu \in \mathbb{R}^D$ and covariance matrix $\Sigma \in \mathbb{R}^{D \times D}$.

Necessary Condition for Gaussianity (Cramér–Wold)

Given a D-dimensional Gaussian distribution in $\mathbb{R}^D$, the projection onto any unit vector on the hypersphere $\mathbb{S}^{D-1}$ must form a 1-dimensional Gaussian distribution.

More formally:

Let $X \sim \mathcal{N}(\mu, \Sigma)$ be a random vector in $\mathbb{R}^D$.

Then, for every unit vector $a \in \mathbb{S}^{D-1}$ (i.e. $\|a\| = 1$),

$$\langle a, X \rangle \sim \mathcal{N}(a^T \mu, a^T \Sigma a) \quad (\text{a 1-D Gaussian})$$

Distribution Equivalence (Cramér–Wold)

Two multivariate distributions are equal if and only if all of their 1-dimensional projections are equal.

More formally:

Let X, Y be random vectors in $\mathbb{R}^D$.

If, for every unit vector $a \in \mathbb{S}^{D-1}$ (i.e. $\|a\| = 1$),

$$\langle a, X \rangle =^d \langle a, Y \rangle \quad (\text{equal in distribution})$$

then

$$X =^d Y \quad (\text{equal in distribution})$$

This avoid the **curse of dimensionality!**

SIGReg

1. Sample random directions

$$A = [a_1, a_2, \dots, a_M]$$

2. Project data onto these directions

$$z^T \cdot A = [u_1, u_2, \dots, u_B]$$

3. Compute empirical characteristic function (Fourier transform of the empirical distribution)

$$\hat{\varphi}(t) = \frac{1}{B} \sum_{b=1}^B e^{jtu_b}$$

4. Target condition: isotropic Gaussian check

$$\varphi_{\mathcal{N}(0,1)}(t) = \hat{\varphi}(t) \quad \forall t,$$

then the latent space follows an isotropic Gaussian distribution.

5. Divergence between distributions (intractable in closed form)

$$\int |\hat{\varphi}(t) - \varphi_{\mathcal{N}(0,1)}(t)|^2 w(t) dt$$

6. Empirical characteristic function (ECF) approximation

$$\begin{aligned} \hat{\varphi}(t) &= \frac{1}{B} \sum_{b=1}^B e^{jtu_b} \\ &= \frac{1}{B} \sum_{b=1}^B \cos(tu_b) + j \frac{1}{B} \sum_{b=1}^B \sin(tu_b) \\ &= \text{Re}(\hat{\varphi}(t)) + j \text{Im}(\hat{\varphi}(t)) \end{aligned}$$

7. Discrete approximation: Epps-Pulley statistic

$$\begin{aligned} \text{err}[t] &= |\hat{\varphi}(t) - \varphi_{\mathcal{N}(0,1)}(t)|^2 \\ &= \left(\text{Re}(\hat{\varphi}(t)) - e^{-t^2/2} \right)^2 + \text{Im}(\hat{\varphi}(t))^2 \end{aligned}$$

$$\text{EP} = \sum_{t=-5}^5 \text{err}[t] w[t] \Delta t$$

8. Final SIGReg loss: average over all M directions

$$\mathcal{L}_{\text{SIGReg}} = \frac{1}{M} \sum_{m=1}^M \text{EP}_m$$

Global Views vs Local Views

The idea of the split is to force the model to learn **object composition**.

The only difference between the two is the random crop scale:

1. **Global views**: scale varies in the range [0.3, 1.0]
2. **Local views**: scale varies in the range [0.05, 0.3]

Global views help to capture the **semantics of the scene** whereas **local views** excel at enhancing **high-frequency features** for object boundaries.

Global Views vs Local Views

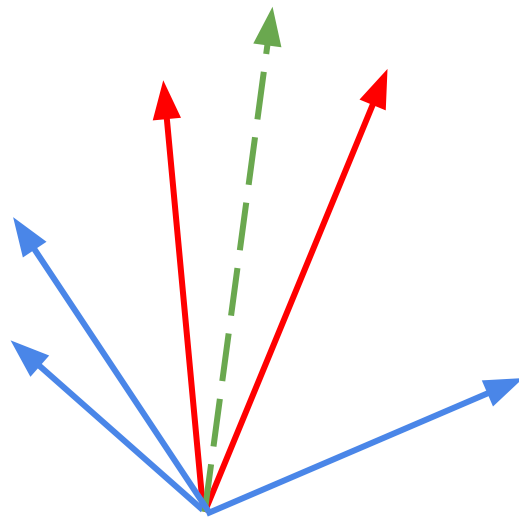
This distinction is key in computing a component of the **LeJEPa loss**, namely **prediction loss**, whose goal is to optimize the prediction of global views:

$$\mathcal{L}_{pred}(\{z_{n,v}\}_{v=1}^V) = \frac{1}{V} \sum_{v=1}^V \|\mu_n - z_{n,v}\|_2^2$$

where $\mu_n = \frac{1}{V_g} \sum_{v=1}^{V_g} z_{n,v}$.

View Invariance in Practice

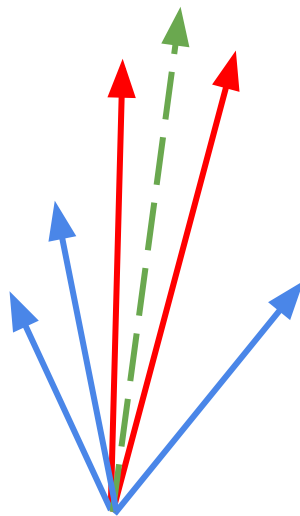
Invariance loss
pulls all these
vectors close



- global views
- local views
- mean of global views

View Invariance in Practice

Invariance loss
pulls all these
vectors close



- global views
- local views
- mean of global views

The Cost Function

$$J(\theta) = \text{EmpiricalRisk}(\theta) + c \text{ Regularizer}(\theta)$$

We can write it as:

$$J(\theta) = (1 - \lambda) \text{EmpiricalRisk}(\theta) + \lambda \text{Regularizer}(\theta) \quad \text{with } \lambda \in [0,1], c = \lambda / (1-\lambda)$$

EmpiricalRisk is the average invariance loss.

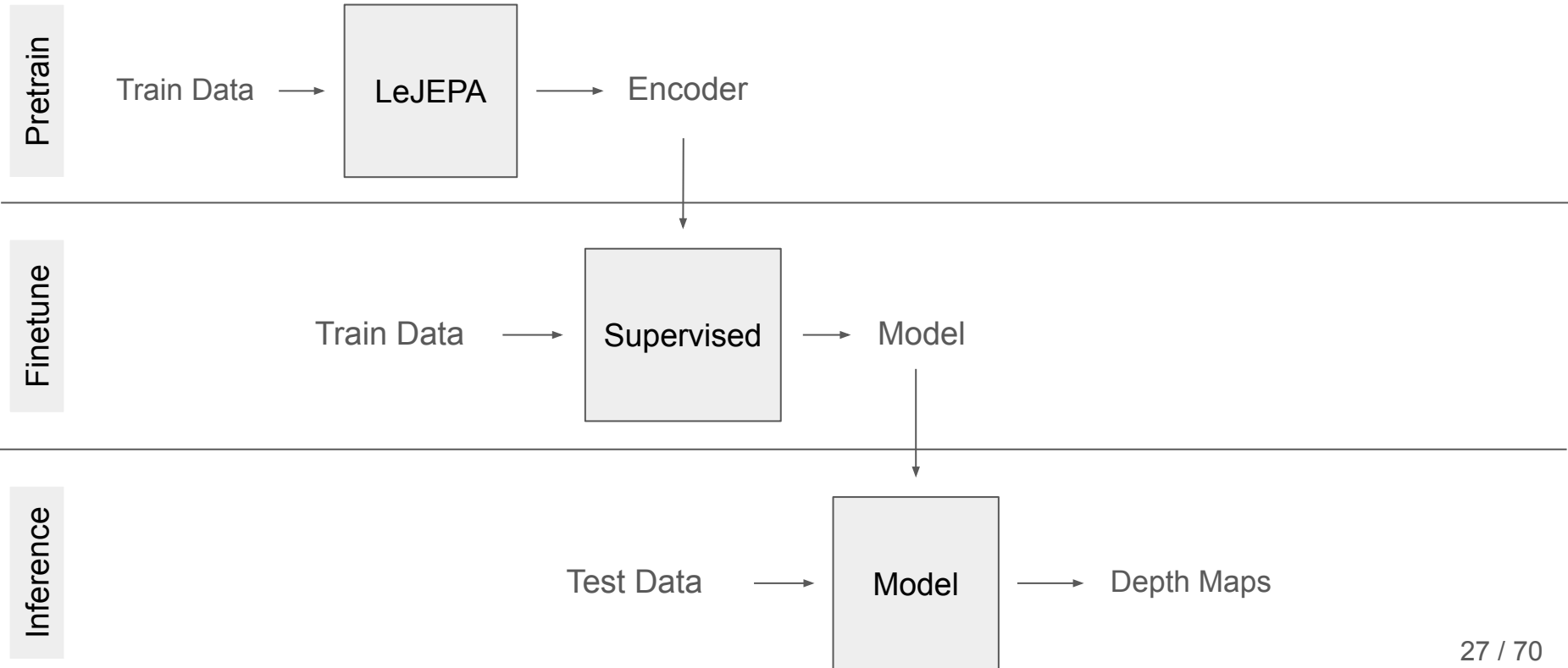
$$\mathcal{L}_{pred}(\{z_{n,v}\}_{v=1}^V) = \frac{1}{V} \sum_{v=1}^V \|\mu_n - z_{n,v}\|_2^2$$

Regularizer is SIGReg.

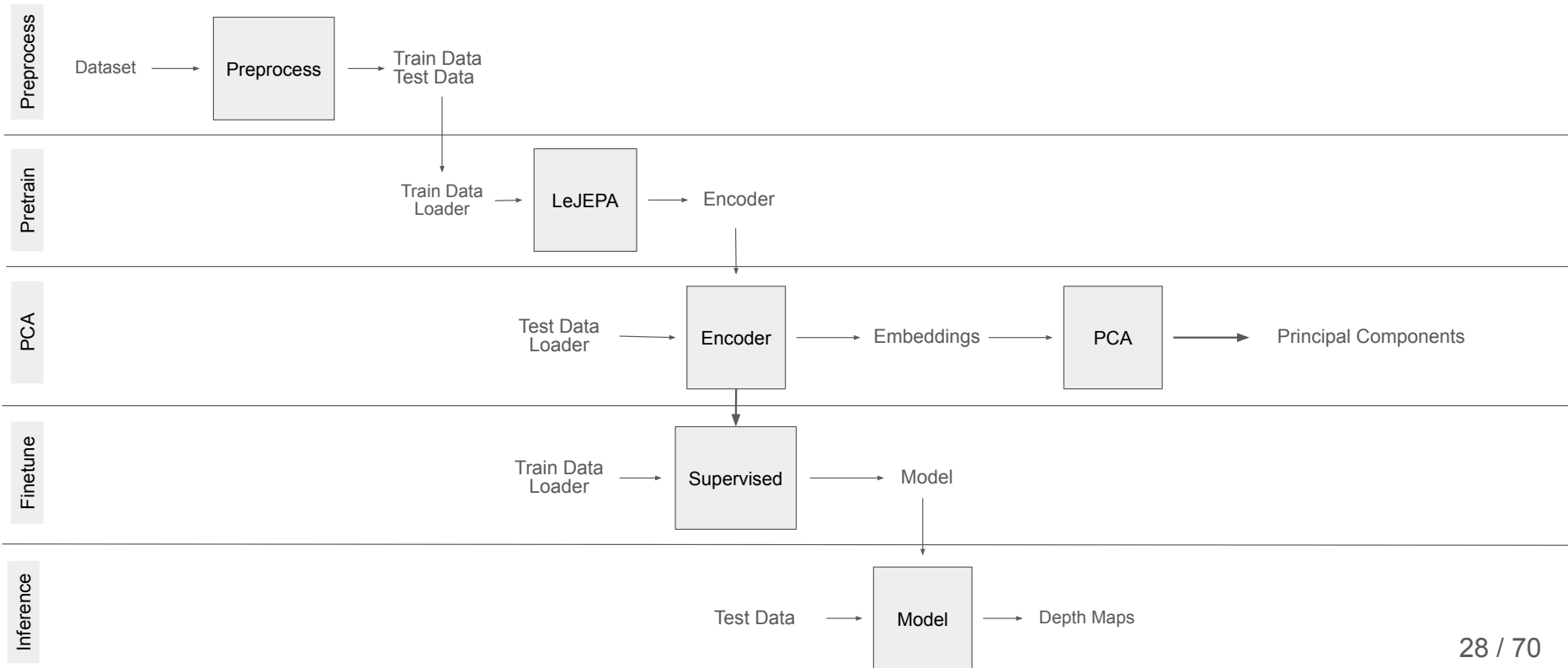
$$\mathcal{L}_{\text{SIGReg}} = \frac{1}{M} \sum_{m=1}^M \text{EP}_m$$

4. Setup

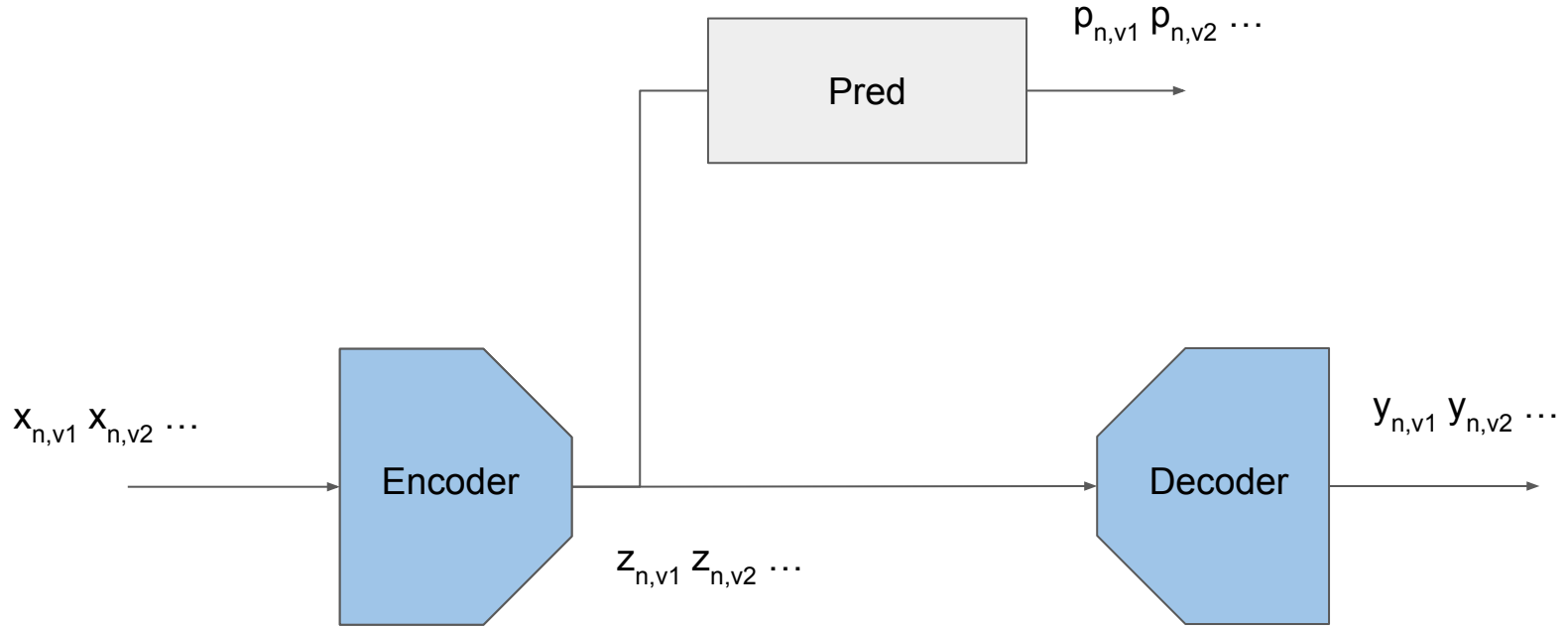
Ideal Setup



Actual Setup



Architecture



Reproducibility

TORCH.MANUAL_SEED(3407) IS ALL YOU NEED

torch.manual_seed(3407) is all you need: On the influence of random seeds in deep learning architectures for computer vision

David Picard

LIGM, École des Ponts, 77455 Marnes la vallée, France

DAVID.PICARD@ENPC.FR

Faster Training

- Data preprocessing
- Concurrent dataloaders
- `torch.compile()`
- Pin memory
- Prefetch of batches
- Batchsize to max out VRAM
- Checkpointing

5. Ablation

Augmentation

Augmentation is key to force the model to reason about **spatial semantics**.

Effect Policy	Mirror	Crop	Color Jitter	Grayscale	Gaussian Blur	Solarize	Vertical Flip	Channel Swap	Shifting Strategy	Gamma	Brightness	Color	Depth Shift
DINO	✓	✓	✓	✓	✓	✓	x	x	x	x	x	x	x
METER ^[2]	✓	✓	x	x	x	x	✓	✓	✓	✓	✓	✓	✓

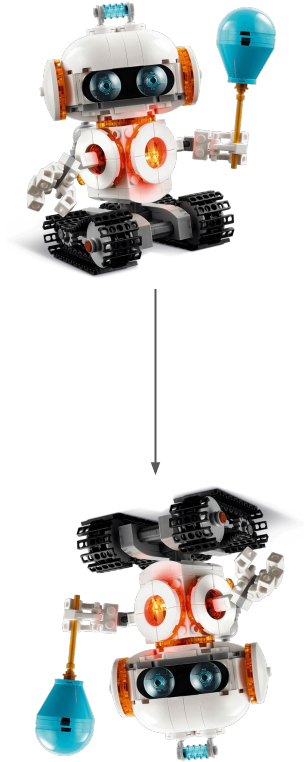
We have also tested a non-deterministic mixed approach by branching between the two policies with probability p .

What makes a good augmentation?

It depends!

One may argue that vertical flip makes no sense.

If our robot ever gets flipped over, firstly, the depth perception is not the main issue, secondly, we have always a position sensor. We can use that to rectify the image.



What makes a good augmentation?

Other may argue that blurring removes depth related info, indeed, models may not be able to predict depth if the blur is too aggressive.

But if the augmentations are too conservative, there isn't much to learn!

Photometric transformations are good.

Channel swaps too, because they are lossless.

Hyperparameters

- η : learning rate
- B : batch size
- Number of slices: number of random directions for SIGReg
- λ : balances the trade-off between the prediction and isotropic gaussian terms
- V : number of views
- V_g : number of global views

6. Probing and Metrics

Online Probing

Online probing is useful to get a live feedback about the training. If a run is degenerating, we can use the linear probe to catch it, and stop the training, saving resources.

In LeJEPA, they used a simple linear probe

`EMBEDDING_DIM` $\rightarrow$ `NUM_CLASSES`

For MDE, we used a linear layer with shape

`EMBEDDING_DIM` $\rightarrow$ (48, 64)

And also a 2-layer non-linear probe with GELU activations

Effective Rank (erank)

Let $Z \in \mathbb{R}^{N \times d}$ be N latent vectors (embeddings) stacked as rows, each of dimension d . Compute singular values $\sigma_1, \dots, \sigma_d$ of Z (or of its covariance matrix), normalize to $p_i = \sigma_i / \sum \sigma_j$, then:

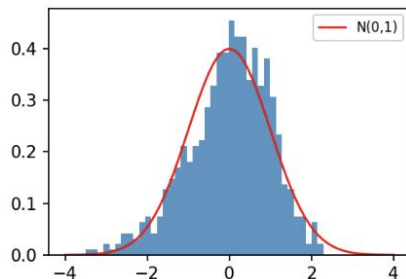
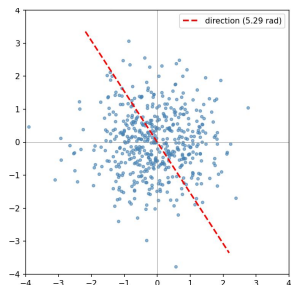
$$\mathbf{erank}(Z) = \exp(- \sum_i p_i \log p_i)$$

It measures how evenly the matrix's “*energy*” is spread across its singular directions. A **healthy**, non-collapsed representation has **erank close to d** .

Random Projections

Exactly the same idea from the necessary condition of Cramer-Wold. Project embeddings onto a random direction, and measure the gaussianity of the distribution via:

- Skew = $E[(X-\mu)^3] / \sigma^3$ (asymmetry)
- Excess Kurtosis = $E[(X-\mu)^4]/\sigma^4 - 3$ (tailedness)
- KS = $\sup | \text{empirical_cdf}(x) - \text{normal_cdf}(x) |$ (cdf distance)



skew: -0.50
exkurt: 0.16
KS: 0.047

Spearman Rank Correlation (ρ)

How well two variables track each other in rank order, assuming only a monotonic relation (no assumptions on linearity). We use it to measure the correlation between a PC and the ground truth.

Score ranges from -1 to $+1$; we take the absolute value, since a component can be "inversely related" to depth and still be just as informative

Chosen over standard correlation because it only checks for a consistent up/down trend, which is the right test for whether a component captures depth at all

Mutual Information

Let X be a random variable, let $H(X)$ be the entropy of X . By observing another random variable Y , the remaining uncertainty about X is the conditional expectation $H(X|Y)$.

The mutual information is simply the difference of the two. It quantifies **how much uncertainty about X it removes the observation of Y :**

$$I(X;Y) = H(X) - H(X|Y)$$

IoU

It answers to a very simple question: of the total space occupied by two sets, how much of it is shared?

$$IoU = \frac{|A| \cap |B|}{|A| \cup |B|} = \frac{|A| \cap |B|}{|A| + |B| \setminus |A \cap B|}$$

In particular for fg/bg IoU captures **how well** the model **captured** the **object of interest without bleeding** into the surrounding space, via a confusion matrix.

$$IoU_{fg/bg} = \frac{TP}{TP + FP + FN}$$

Offline Ridge Probe

The optimal linear decoder for L_2 loss and L_2 regularization is the Ridge regressor, which has a closed form optimal solution

$$\theta_{\text{probe}} = (Z^T Z + \lambda I)^{-1} Z^T Y$$

We use it as offline probe to evaluate how much information is embedded in the latent space (how much variance of the ground truth is explained by the decoder outputs)

$$R^2 = 1 - \frac{SS_{res}}{SS_{tot}}$$

$$SS_{res} = \sum_i (y_i - f_i)^2 = \sum_i e_i^2$$
$$SS_{tot} = \sum_i (y_i - \bar{y})^2$$

Metrics

- **RMSE**: average magnitude of the error across all prediction.
- **REL**: error scaled proportionally to the true depth.
- δ_1 : percentage of predictions that are "close enough" to the ground truth.

$$RMSE^{[2]} = \sqrt{\frac{1}{|n|} \sum_{i \in n} \|y_i - \hat{y}_i\|^2}$$

$$REL^{[2]} = \frac{1}{|n|} \sum_{i \in n} \frac{|y_i - \hat{y}_i|}{y_i}$$

$$\delta_1^{[2]} = \frac{1}{|n|} \sum_{i \in n} \max \left(\frac{y_i}{\hat{y}_i}, \frac{\hat{y}_i}{y_i} \right) < thr$$

7. Results

PCA

Through PCA we perform zero-shot geometric probing to see if LeJEPA is able to **detect edge discontinuities**, **without** any kind of **finetuning**, namely **distinguish** the **foreground** from the **background**.

We performed the analysis on three encoders:

- LeJEPA
- METER
- Random

PCA

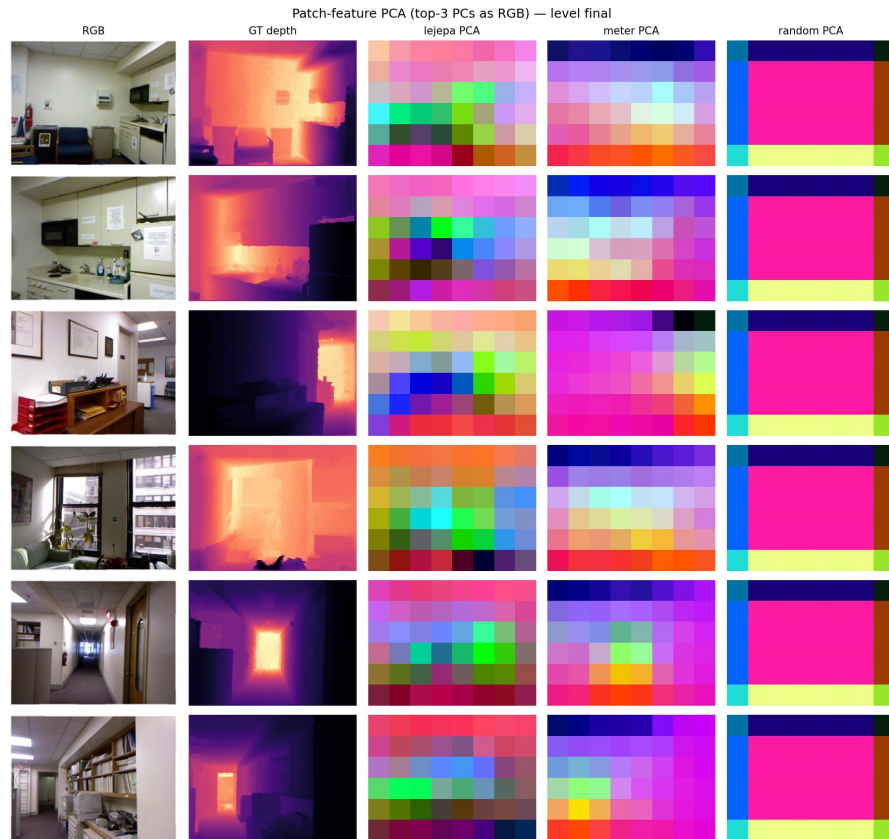
Encoder	Level	erank (patch) $\uparrow$	top-3 var $\downarrow$	best-PC $ \rho \uparrow$	fg/bg IoU $\uparrow$	MI best $\uparrow$	probe $R^2 \uparrow$	probe RMSE (m) $\downarrow$
LeJEPA	y2	20.1/48	0.38	0.287	0.395	0.015	0.037	1.307
LeJEPA	y3	22.1/64	0.37	0.381	0.413	0.033	0.081	1.263
LeJEPA	final	36.1/160	0.34	0.561	0.410	0.122	0.305	1.072
METER	final	14.6/160	0.63	0.736	0.501	0.625	0.788	0.593
Random	final	4.2/160	0.89	0.466	0.433	0.165	0.111	1.213

- **LeJEPA** maintains a **diverse, higher-dimensional feature space** compared to METER.
- Despite LeJEPA best-PC increases significantly going deeper in the network, it does not reach METER.
- **METER** absolutely **dominates** the **supervised linear probe**.

PCA

PCA allows us to **visualize** the **top three components** as **RGB values**, hence visually inspect the internal representation of the data.

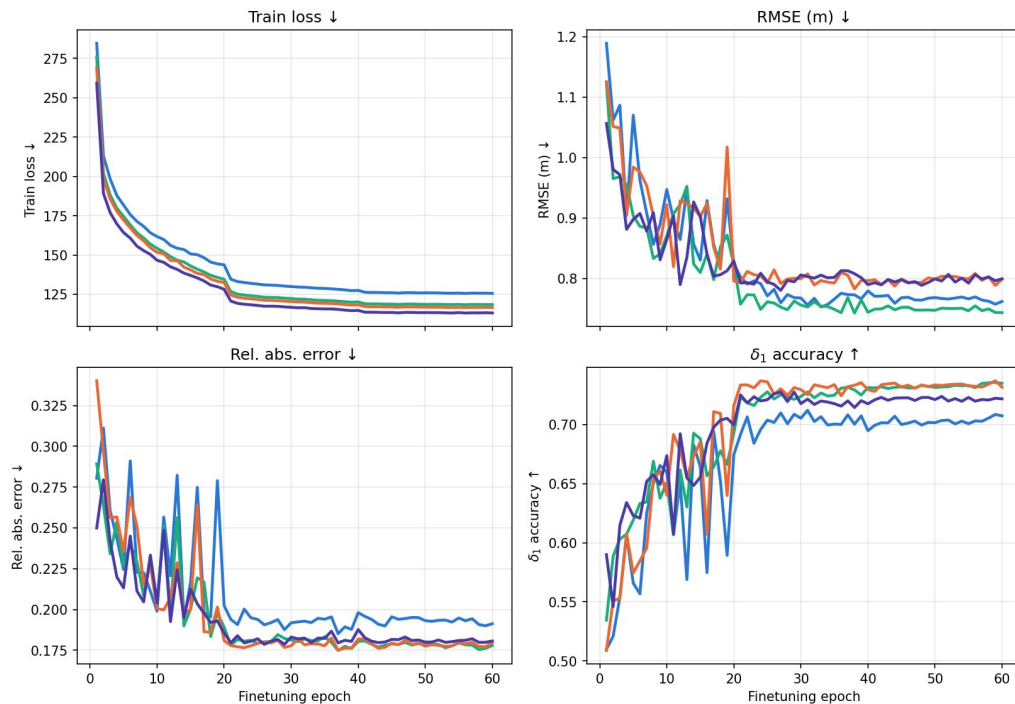
As per fg/bg IoU, LeJEPA seems to somewhat segment the foreground from the background.



Finetuning with METER augmentation

METER finetuning: effect of LeJEPa pretraining with METER augmentation

no pretrain 5 ep pretrain 10 ep pretrain 20 ep pretrain

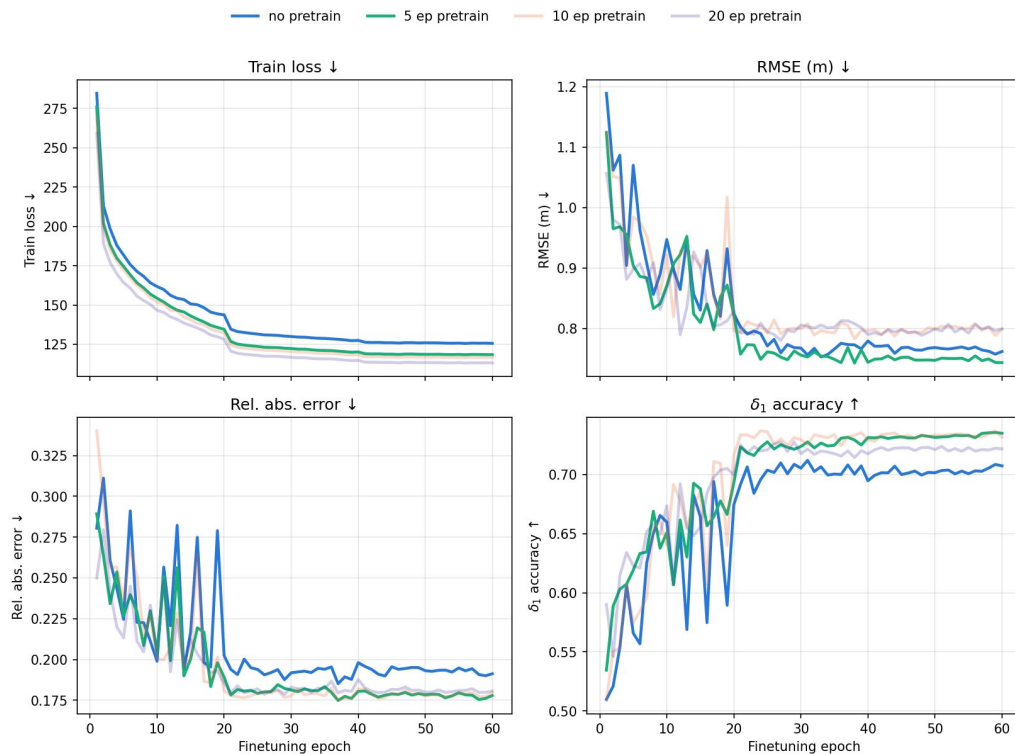


Setting	RMSE ↓	REL ↓	δ_1 ↑
No pretrain	0.628	0.194	0.712
5 epochs	0.630	0.192	0.708
10 epochs	0.645	0.194	0.701
20 epochs	0.665	0.206	0.685

Best model over validation set throughout 60 epochs

Finetuning with METER augmentation

METER finetuning: effect of LeJEPa pretraining with METER augmentation



Setting	RMSE ↓	REL ↓	δ_1 ↑
No pretrain	0.628	0.194	0.712
5 epochs	0.630	0.192	0.708
10 epochs	0.645	0.194	0.701
20 epochs	0.665	0.206	0.685

Best model over validation set throughout 60 epochs

Pretraining Epoch Ablation

Augmentation	epochs	erank (patch) $\uparrow$	top-3 var $\downarrow$	best-PC $ \rho \uparrow$	fg/bg IoU $\uparrow$	MI best $\uparrow$	probe $R^2 \uparrow$	probe RMSE (m) $\downarrow$
LeMETER	60	68.6 /160	0.25	0.592	0.441	0.184	0.302	1.075
LeMETER	20	55.2/160	0.27	0.589	0.416	0.179	0.302	1.074
METER	60	41.8/160	0.35	0.625	0.444	0.161	0.269	1.099
METER	20	30.0/160	0.44	0.587	0.467	0.121	0.292	1.082

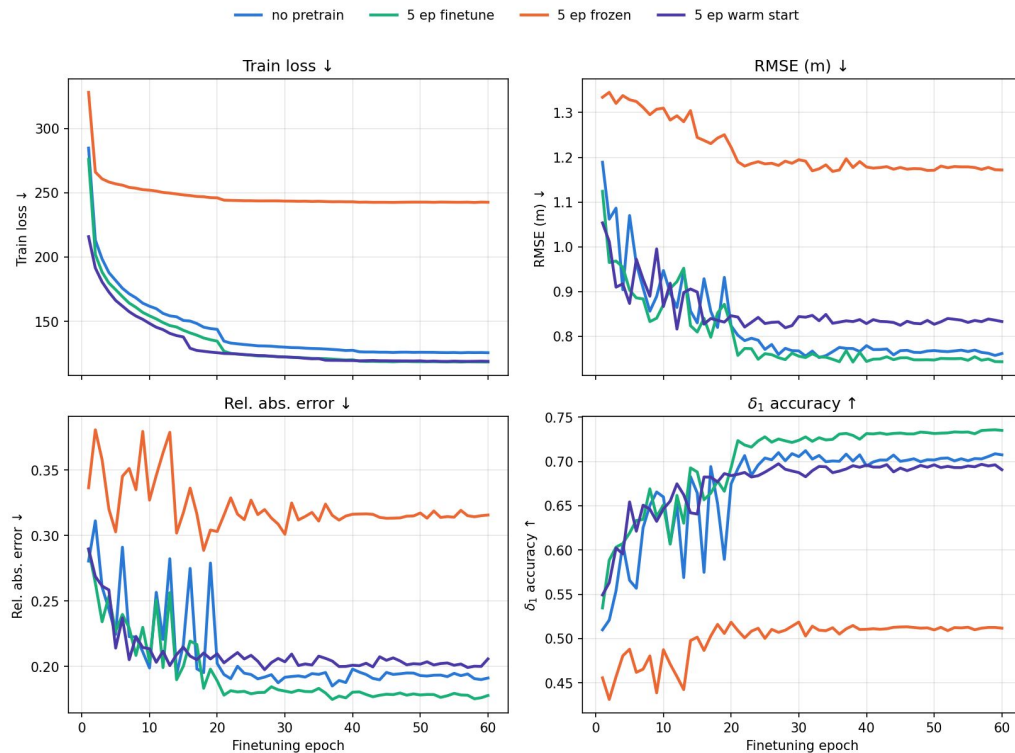
From the results above we can clearly see that **training** the encoder for **more epochs does not imply better performance**.

LeMETER has the same R^2 and RMSE, whereas METER actually worsened its R^2 and RMSE.

It is clear that training for more epochs increase the isotropy of the latent space, which ultimately does not lead to good results though.

Finetuning Schedule Ablation

METER supervised training: effect of different training schedules



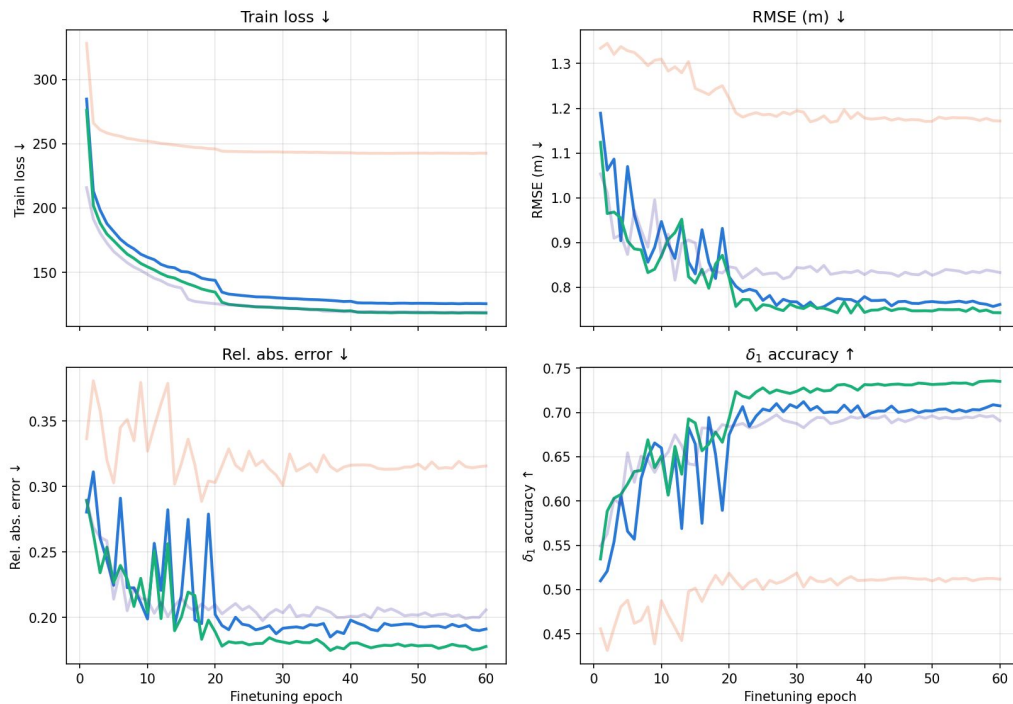
Schedule	RMSE ↓	REL ↓	δ_1 ↑
Fine-tune	0.630	0.192	0.708
Warm start	0.684	0.227	0.670
Freeze encoder	1.054	0.352	0.453

Best model over validation set throughout 60 epochs

Finetuning Schedule Ablation

METER supervised training: effect of different training schedules

no pretrain 5 ep finetune 5 ep frozen 5 ep warm start



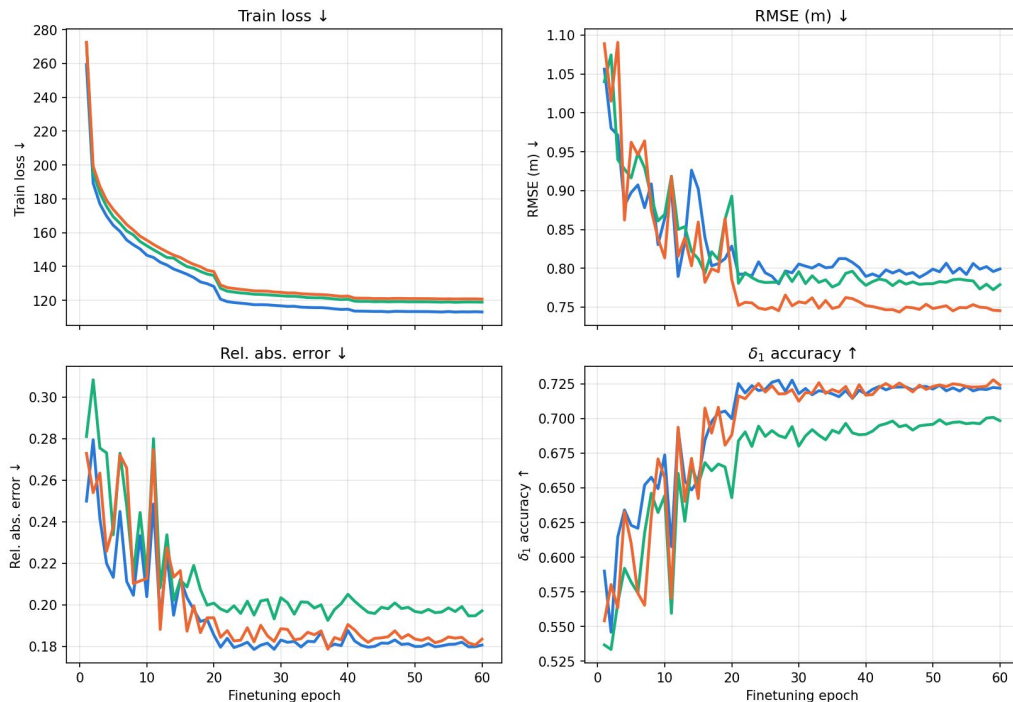
Schedule	RMSE ↓	REL ↓	δ_1 ↑
Fine-tune	0.630	0.192	0.708
Warm start	0.684	0.227	0.670
Freeze encoder	1.054	0.352	0.453

Best model over validation set throughout 60 epochs

Finetuning Augmentation Ablation

METER finetuning: effect of pretraining augmentation strategy

METER DINO LeMETER



Augmentation	RMSE ↓	REL ↓	δ_1 ↑
LeMETER	0.636	0.195	0.705
DINO	0.634	0.191	0.710
METER	0.630	0.192	0.708

Best model over validation set throughout 60 epochs

8. Conclusion

Why did the pretraining not work?

The **pretraining did work**, maybe not as well as we would have wanted. From unlabeled data, we were able to get a classifier 3x better (probe R^2) than a random one.

Real data usually lives on a **low-dimensional manifold** inside a **higher-dimensional embedding space**, and forcing isotropy across all dimensions works against letting the model concentrate signal into a compact low-rank subspace.

Consequence: isotropic-Gaussian targets need more ambient dimensions to represent the same information.

Why did the pretraining not work?

It also comes down to the difference between **invariance** and **equivariance**.

For **classification**, we want **invariance** to all transformations.

But in **regression**, we may want **equivariance** to some transformations.

E.g. invariance to 50% crop is good for classification, but horrendous for regression.

We want embedding z statistically dependent on transformation v . Not "equal to a function of a ," not "correlated in a particular direction".

Future Work: Equivariant Framework Sketch

$$\mathcal{J}(\theta) = \mathcal{L}_{\text{pred}}(\theta) + \lambda \mathcal{R}_{\text{equiv}}(\theta)$$

$$\mathcal{L}_{\text{pred}}(\theta) = \frac{1}{B(B-1)} \cdot \frac{1}{V} \sum_{i \neq j} \sum_{v=1}^V \ell(a_i^v, a_j^v)$$

Equivariant Prediction Loss

$$\ell(a_i^v, a_j^v) = 1 - \frac{\langle a_i^v, a_j^v \rangle}{\|a_i^v\| \|a_j^v\|}$$

with drift vectors

$$z_i = f_{\theta}(x_i)$$

$$z_i^v = f_{\theta}(\text{transform}_v(x_i))$$

$$a_i^v = z_i - z_i^v$$

With a proper regularizer to avoid representation collapse (e.g. $f(\text{transform}_v(x)) = f(x) + c_v$), this could be applicable, although being $O(B^2)$

Conclusions

LeJEPA shows results for classification task.

We tried applying it to a regression task.

LeJEPA **out-of-the-box** gives **no measurable benefit** for MDE at single-GPU budgets:

- no significant boost in accuracy
- **identical, if not worse**, training convergence speed

9. Limitations

Limitations

For our training we used a Nvidia **RTX 4090 SUPER**, which has **24GB of VRAM**.

LeJEPA provided optimal hyperparameters used for models of 300M+ parameters.

	Actual Value	Optimal Value
Number of slices	256	1024 ^[1]
Batch size	64	128 ^[1]
Pretrain epochs	20	100 ^[1]

Limitations

In addition to the complexity of the project itself we had to face some challenges while trying to understand our references:

- Inconsistencies in the LeJEPA paper (e.g. loss & sigreg implementations)
- METER's public code on GitHub not working as expected out of the box
- Figuring out why NYU was being normalized with ImageNet mean and variance

Vibecoding red flags 🚩 #7

[Open](#)

JGalego opened on Nov 14, 2025

Last edited by JGalego ▾ ⋮

I've been going through the code and documentation and I couldn't help but notice that this repository exhibits several tell tale signs of an early vibe coded prototype:

- non-existing PyPI packages (lejepa)
- `setup.py` declares the package as "deepstats" not "lejepa"
- missing training scripts (`je.py`) and configs
- duplicated classes (`EppsPulley`)
- outdated instructions (`num_points` , instead of `n_points`)
- the use of long-deprecated functions like Matplotlib's `get_cmap`
- multiple uses of the word "comprehensive"
- &c.

Care to comment?



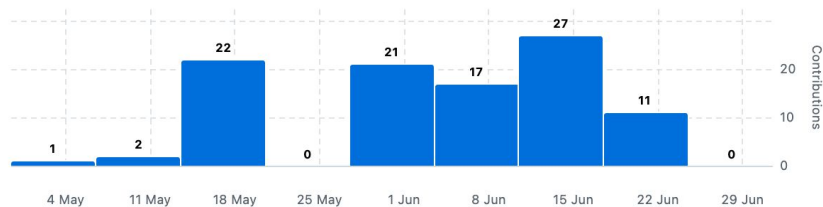
Limitations

In order to achieve the presented results many iterations were required, reaching approximately **~100h GPU**.

Also due to the high number of ablations we tested, the [LeMETER](#) GitHub repository has some flows external to the [main.py](#) script.

Commits over time

Weekly from 28 mar 2026 to 28 giu 2026

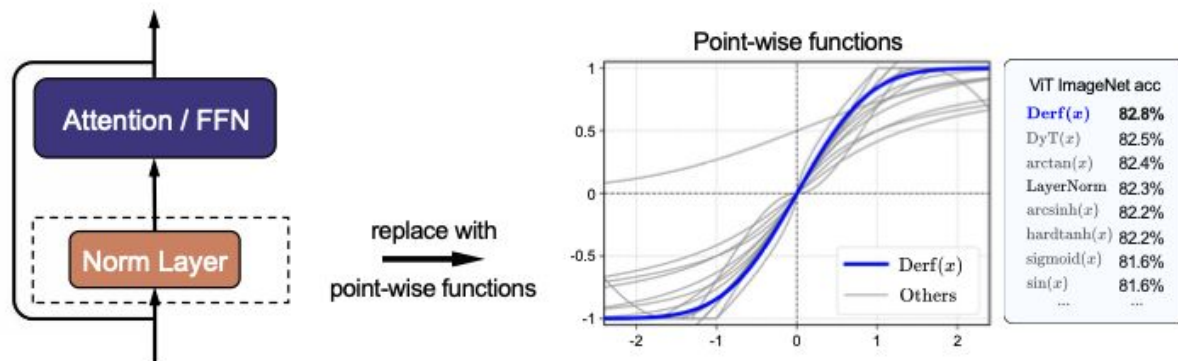


#2

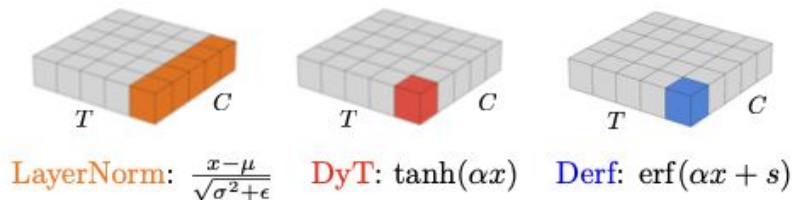
f9fe072 · 2 hours ago 106 Commits

10. Gaussian METER

Pointwise Normalization



a) We search point-wise functions of different shapes as norm layer replacement.



b) Formulation of LayerNorm (LN), DyT, and Derf (ours).

method	ViT acc (↑)	DiT FID (↓)	DNA acc (↑)
LN	82.3%	45.91	86.9%
DyT	82.5%	45.66	86.9%
Derf	82.8%	43.94	87.3%

c) Performance across domains.

Gaussian METER (xxs)

Norm	Act.	RMSE ↓	REL ↓	δ_1 ↑	FPS ↑	
					GPU	CPU
LayerNorm	ReLU	0.628	0.194	0.712	238.86	104.0 ± 20
LayerNorm	GELU	0.617	0.186	0.724	238.05	88.6 ± 20
DyERF	ReLU	0.625	0.192	0.717	239.82	113.2 ± 20
DyERF	GELU	0.605	0.184	0.731	237.80	102.8 ± 20

References

- [1]. Randall Balestriero, Yann LeCun. “LeJEPA: Provable and Scalable Self-Supervised Learning Without the Heuristics”. arXiv, 2025.
- [2]. Lorenzo Papa, Paolo Russo, Irene Amerini. “METER: a mobile vision transformer architecture for monocular depth estimation”. IEEE Transactions on Circuits and Systems for Video Technology, 2023.
- [3] Jiahuan Long, Xin Zhou. “LMDepth: Lightweight Mamba-based Monocular Depth Estimation for Real-World Deployment”. arXiv, 2025
- [4] Mingzhi Chen, Taiming Lu, Jiachen Zhu, Mingjie Sun, Zhuang Liu: “Stronger Normalization-Free Transformers”. arXiv, 2026
- [5] Nathan Silberman, Derek Hoiem, Pushmeet Kohli, Rob Fergus: “Indoor Segmentation and Support Inference from RGBD Images”. ECCV, 2012
- [6] Lorenzo Papa , Edoardo Alati , Paolo Russo, Irene Amerini: “SPEED: Separable Pyramidal Pooling Encoder-Decoder for Real-Time Monocular Depth Estimation on Low-Resource Settings”. IRIS, 2022
- [7] David Picard: “Torch.manual_seed(3407) is all you need: On the influence of random seeds in deep learning architectures for computer vision”. arXiv, 2021
- [8] Diana Wofk, Fangchang Ma, Tien-Ju Yang, Sertac Karaman, Vivienne Sze: “FastDepth: Fast Monocular Depth Estimation on Embedded Systems”. arXiv, 2019
- [9] Ning Zhang, Francesco Nex, George Vosselman, Norman Kerle: “Lite-Mono: A Lightweight CNN and Transformer Architecture for Self-Supervised Monocular Depth Estimation”. arXiv, 2022
- [10] Andrii Litvynchuk, Ivan Livinsky, Anand Ravi, Nima Kalantari, Andrii Tsarov: “EfficientDepth: A Fast and Detail-Preserving Monocular Depth Estimation Model”. arXiv, 2025

Thank you for your attention!

maruf.2052302@studenti.uniroma1.it
natilla.2049557@studenti.uniroma1.it